

Prueba Técnica: Programador Back-End en .NET Core con SQL Server

Objetivo

Evaluar la capacidad para diseñar y exponer una API RESTful en .NET Core conectada a SQL Server, aplicando EF Core, procedimientos almacenados, validaciones, paginación y pruebas automáticas.

Duración estimada

4 horas

Alcance funcional

Construir una API para gestión de Pacientes:

1. Requerimientos de la API

Patients

- POST /api/patients
Crear paciente. Validar duplicados por (DocumentType, DocumentNumber).
- GET /api/patients
Listar con paginación (page, pageSize) y filtros opcionales (name, documentNumber).
- GET /api/patients/{id}
- PUT /api/patients/{id}
Permitir actualización parcial o total (a elección del candidato, pero documentar).
- DELETE /api/patients/{id}

2. Entidades

Diseña una base de datos en SQL Server con una tabla Patients que contenga los siguientes campos:

- Patients
 - PatientId (PK, int, Identity)
 - DocumentType (nvarchar(10))
 - DocumentNumber (nvarchar(20), único)
 - FirstName (nvarchar(80))
 - LastName (nvarchar(80))
 - BirthDate (date)
 - PhoneNumber (nvarchar(20), null)
 - Email (nvarchar(120), null)
 - CreatedAt (datetime2, default GETUTCDATE())

3. Implementación y Conexión a SQL Server:

- Implementa la API conectándola a la base de datos SQL Server creada.
- Utiliza Entity Framework Core para las operaciones CRUD.
- Crea al menos un procedimiento almacenado para obtener los pacientes creado después de una fecha específica.

4. Pruebas:

- Implementa pruebas unitarias para al menos dos endpoints utilizando un framework de pruebas como xUnit o NUnit.

5. Documentación:

Incluye un archivo README.md con instrucciones para la instalación y configuración de la API, así como para la creación de la base de datos.

Describe brevemente la arquitectura de la API y las razones detrás de tus decisiones técnicas.

6. Entrega:

Subir el proyecto a un repositorio en GitHub y compartir el enlace.

Criterios de Evaluación:

- Estructura y calidad del código.
- Diseño de la base de datos y uso de procedimientos almacenados.
- Correcta implementación y funcionamiento de la API.
- Cobertura y calidad de las pruebas unitarias.
- Documentación clara y detallada.

Prueba de Desarrollo: Programador Front-End en Angular

Objetivo: Evaluar las habilidades del candidato en el desarrollo de una aplicación web utilizando Angular, así como su capacidad para trabajar con APIs y realizar pruebas unitarias.

Duración estimada: 6 horas

Alcance funcional

Construir una app con 1 módulo:

1. Pacientes

- Listado paginado/filtrable.
- Opciones de Crear / Editar / Eliminar.
- Ver detalle de un paciente

API Backend:

- Consumir la API creada anteriormente

Requerimientos de la UI

Stack y setup

- **Angular 16**
- **PrimeNG**
- **Services** por recurso, con **HttpClient** y **interceptor** para:
 - Manejo de errores global (toast/dialog).
 - (Opcional) inclusión de token (si habilitas Auth extra).

Pacientes

- Vista **Lista**:
 - Tabla paginada (client o server-side) con: Documento, Nombre, Apellidos, Email, Teléfono, Fecha creación.
 - Filtros: name (contiene), documentNumber (exacto).
 - Acciones: Ver, Editar, Eliminar.
- Vista **Formulario** (crear/editar):
 - Campos obligatorios: DocumentType, DocumentNumber, FirstName, LastName, BirthDate.
 - Validaciones en UI + mensajes amigables (ej. duplicado por documento).
- Vista **Detalle**:
 - Mostrar datos del paciente y **lista de citas** asociadas (solo lectura).

Reportes (simple) (Adicional no obligatorio)

- Modal que genere excel con filtro de fecha de creación, para consultar todos los pacientes creados después de esa fecha.
- Botón “**Exportar CSV**”

UX / Estándares

- **Feedback** visual en operaciones (spinner, botón en loading, deshabilitar submit).
- **Confirmación** antes de eliminar.
- **Manejo de errores:** mostrar message y, si existen, details[].

Pruebas Unitarias:

Implementa al menos 3 pruebas unitarias utilizando Jasmine y Karma para los componentes de la aplicación.

Documentación:

Incluye un archivo README.md con instrucciones para la instalación y ejecución de la aplicación.

Explica brevemente la arquitectura de la aplicación y las decisiones técnicas que tomaste durante el desarrollo.

Entrega: Subir el proyecto a un repositorio en GitHub y compartir el enlace.

Criterios de Evaluación:

- Calidad del código (estructura, claridad, comentarios).
- Correcta integración con la API RESTful.
- Funcionamiento completo de la aplicación según los requisitos.
- Cobertura y calidad de las pruebas unitarias.
- Documentación clara y concisa.